Corrigé du TP1 – Le catalogue du disquaire

React Native · Une solution complète, fichier par fichier

Voici une solution complète du TP1. Chaque bloc est un **fichier entier** : copiez-le dans le fichier indiqué par son titre. Comparez-le avec le vôtre plutôt que de le recopier : une autre solution peut être tout aussi juste.

> **Pour lancer la solution**
>
> Partez d'un projet créé comme à l'étape 0 (`create-expo-app` , `reset-project` , `"output": "single"`), puis copiez les fichiers dans l'ordre de cette page. Vérifiez avec `npx tsc --noEmit` , puis lancez `npm start` .

1 Le modèle et les données

Le type `Genre` est une union : une faute de frappe dans un genre est refusée par TypeScript. `GENRES` sert à afficher les boutons du filtre.

src/models/album.ts

```ts
export type Genre = "rock" | "jazz" | "pop" | "hip-hop" | "electro";

export type Album = {
  id: string;
  title: string;
  artist: string;
  year: number;
  genre: Genre;
  price: number;
  inStock: boolean;
  coverUrl: string;
};

export const GENRES: Genre[] = [
  "rock",
  "jazz",
  "pop",
  "hip-hop",
  "electro",
];
```

Les données sont celles de l'énoncé. Le type `Album[]` fait vérifier chaque album.

src/data/albums.ts

```ts
import { Album } from "@/models/album";

export const ALBUMS: Album[] = [
  {
    id: "a1",
    title: "Kind of Blue",
    artist: "Miles Davis",
    year: 1959,
    genre: "jazz",
    price: 24.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/100/400/400",
  },
  {
    id: "a2",
    title: "Random Access Memories",
    artist: "Daft Punk",
    year: 2013,
    genre: "electro",
    price: 34.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/101/400/400",
  },
  {
    id: "a3",
    title: "Rumours",
    artist: "Fleetwood Mac",
    year: 1977,
    genre: "rock",
    price: 27.5,
    inStock: false,
    coverUrl: "https://picsum.photos/id/102/400/400",
  },
  {
    id: "a4",
    title: "To Pimp a Butterfly",
    artist: "Kendrick Lamar",
    year: 2015,
    genre: "hip-hop",
    price: 32,
    inStock: true,
    coverUrl: "https://picsum.photos/id/103/400/400",
  },
  {
```

```
    id: "a5",
    title: "Thriller",
    artist: "Michael Jackson",
    year: 1982,
    genre: "pop",
    price: 22.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/104/400/400",
  },
  {
    id: "a6",
    title: "A Love Supreme",
    artist: "John Coltrane",
    year: 1965,
    genre: "jazz",
    price: 26,
    inStock: true,
    coverUrl: "https://picsum.photos/id/106/400/400",
  },
  {
    id: "a7",
    title: "OK Computer",
    artist: "Radiohead",
    year: 1997,
    genre: "rock",
    price: 29.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/107/400/400",
  },
  {
    id: "a8",
    title: "Discovery",
    artist: "Daft Punk",
    year: 2001,
    genre: "electro",
    price: 31.5,
    inStock: false,
    coverUrl: "https://picsum.photos/id/108/400/400",
  },
  {
    id: "a9",
    title: "Illmatic",
    artist: "Nas",
    year: 1994,
    genre: "hip-hop",
    price: 25,
```

```
    inStock: true,
    coverUrl: "https://picsum.photos/id/109/400/400",
  },
  {
    id: "a10",
    title: "L'Homme à tête de chou",
    artist: "Serge Gainsbourg",
    year: 1976,
    genre: "pop",
    price: 23.5,
    inStock: true,
    coverUrl: "https://picsum.photos/id/110/400/400",
  },
  {
    id: "a11",
    title: "Moanin'",
    artist: "Art Blakey",
    year: 1958,
    genre: "jazz",
    price: 21.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/111/400/400",
  },
  {
    id: "a12",
    title: "Homework",
    artist: "Daft Punk",
    year: 1997,
    genre: "electro",
    price: 28.9,
    inStock: true,
    coverUrl: "https://picsum.photos/id/112/400/400",
  },
];
```

2 La carte d'un album

La mise en forme des prix sert à la carte et au total du panier : elle a son propre fichier.

src/models/price.ts

```ts
// Mise en forme des prix, partagée par la carte et le total du panier
export const formatPrice = (price: number) =>
  new Intl.NumberFormat("fr-FR", {
    style: "currency",
    currency: "EUR",
  }).format(price);
```

La carte ne reçoit que des props. Elle affiche l'album et signale l'appui sur le bouton du panier, sans rien décider elle-même.

src/components/album-card.tsx

```tsx
import {
  Image,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { formatPrice } from "@/models/price";
import { Album } from "@/models/album";

type AlbumCardProps = {
  album: Album;
  inCart: boolean;
  onToggleCart: (id: string) => void;
};

export function AlbumCard({
  album,
  inCart,
  onToggleCart,
}: AlbumCardProps) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: album.coverUrl }} style={styles.cover} />
      <View style={styles.body}>
        <Text style={styles.genre}>{album.genre}</Text>
        <Text style={styles.title} numberOfLines={1}>
          {album.title}
        </Text>
        <Text style={styles.artist}>
          {album.artist} · {album.year}
        </Text>
        <Text style={styles.price}>{formatPrice(album.price)}</Text>
        <Pressable
          disabled={!album.inStock}
          onPress={() => onToggleCart(album.id)}
          style={({ pressed }) => [
            styles.button,
            inCart && styles.buttonInCart,
            !album.inStock && styles.buttonDisabled,
            pressed && styles.buttonPressed,
          ]}
        >
```

```
          <Text style={styles.buttonLabel}>
            {!album.inStock
              ? "Épuisé"
              : inCart
                ? "Dans le panier ✓"
                : "Ajouter au panier"}
          </Text>
        </Pressable>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    flex: 1,
    margin: 8,
    backgroundColor: "#fff",
    borderRadius: 12,
    overflow: "hidden",
  },
  cover: { width: "100%", aspectRatio: 1 },
  body: { padding: 12, gap: 2 },
  genre: {
    fontSize: 12,
    fontWeight: "700",
    textTransform: "uppercase",
    color: "#0B7A70",
  },
  title: { fontSize: 17, fontWeight: "700" },
  artist: { fontSize: 14, color: "#5A5F80" },
  price: { fontSize: 16, fontWeight: "600", marginTop: 4 },
  button: {
    marginTop: 10,
    paddingVertical: 10,
    borderRadius: 8,
    alignItems: "center",
    backgroundColor: "#E2F7F4",
  },
  buttonInCart: { backgroundColor: "#2EC4B6" },
  buttonDisabled: { backgroundColor: "#ECEDF3" },
  buttonPressed: { opacity: 0.6 },
  buttonLabel: { fontSize: 15, fontWeight: "600" },
});
```

3 Le hook des colonnes

`useWindowDimensions()` provoque un nouveau rendu quand la fenêtre change de taille : le nombre de colonnes suit sans recharger la page.

src/hooks/use-columns.ts

```ts
import { useWindowDimensions } from "react-native";

export function useColumns(): number {
  const { width } = useWindowDimensions();
  if (width >= 1200) return 4;
  if (width >= 900) return 3;
  if (width >= 600) return 2;
  return 1;
}
```

5 Le filtre par genre

Comme la carte, le filtre est un composant d'affichage : il reçoit le genre choisi et signale chaque appui.

src/components/genre-filter.tsx

```tsx
import {
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
} from "react-native";

import { Genre, GENRES } from "@/models/album";

type GenreFilterProps = {
  selected: Genre | null;
  onSelect: (genre: Genre | null) => void;
};

export function GenreFilter({
  selected,
  onSelect,
}: GenreFilterProps) {
  const options: (Genre | null)[] = [null, ...GENRES];
  return (
    <ScrollView
      horizontal
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={styles.row}
    >
      {options.map((genre) => (
        <Pressable
          key={genre ?? "all"}
          onPress={() => onSelect(genre)}
          style={[
            styles.chip,
            selected === genre && styles.chipSelected,
          ]}
        >
          <Text style={styles.label}>{genre ?? "Tous"}</Text>
        </Pressable>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  row: { gap: 8, paddingHorizontal: 8 },
  chip: {
```

```
    paddingVertical: 6,
    paddingHorizontal: 14,
    borderRadius: 20,
    borderWidth: 1,
    borderColor: "#D9DCEA",
    backgroundColor: "#fff",
  },
  chipSelected: {
    backgroundColor: "#2EC4B6",
    borderColor: "#2EC4B6",
  },
  label: {
    fontSize: 14,
    fontWeight: "600",
    textTransform: "capitalize",
  },
});
```

✓ L'écran du catalogue

Le titre de l'application est défini une fois pour toutes dans le navigateur racine. `headerTransparent: false` rend l'en-tête opaque : sur iOS 26, il est transparent par défaut et l'écran passerait dessous.

src/app/_layout.tsx

```tsx
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{ title: "Disquaire", headerTransparent: false }}
    />
  );
}
```

L'écran assemble tout. Le panier, la recherche et le genre choisi sont des **états**. La liste affichée et le total se **calculent** à partir d'eux à chaque rendu : ce ne sont pas des états. La prop `key` de la `FlatList` la reconstruit quand le nombre de colonnes change.

src/app/index.tsx

```tsx
import { useState } from "react";
import {
  FlatList,
  StyleSheet,
  Text,
  TextInput,
  View,
} from "react-native";

import { AlbumCard } from "@/components/album-card";
import { formatPrice } from "@/models/price";
import { GenreFilter } from "@/components/genre-filter";
import { ALBUMS } from "@/data/albums";
import { useColumns } from "@/hooks/use-columns";
import { Genre } from "@/models/album";

export default function CatalogScreen() {
  const columns = useColumns();
  const [query, setQuery] = useState("");
  const [genre, setGenre] = useState<Genre | null>(null);
  const [cart, setCart] = useState<string[]>([]);

  const toggleCart = (id: string) =>
    setCart((current) =>
      current.includes(id)
        ? current.filter((c) => c !== id)
        : [...current, id],
    );

  const search = query.trim().toLowerCase();
  const visibleAlbums = ALBUMS.filter(
    (album) =>
      (genre === null || album.genre === genre) &&
      (album.title.toLowerCase().includes(search) ||
        album.artist.toLowerCase().includes(search)),
  );

  const total = ALBUMS.filter((album) =>
    cart.includes(album.id),
  ).reduce((sum, album) => sum + album.price, 0);

  return (
    <View style={styles.screen}>
      <View style={styles.container}>
```

```
        <View style={styles.header}>
          <Text style={styles.heading}>Disquaire</Text>
          <Text style={styles.cart}>
            Panier : {cart.length} · {formatPrice(total)}
          </Text>
        </View>
        <TextInput
          value={query}
          onChangeText={setQuery}
          placeholder="Rechercher un album ou un artiste"
          style={styles.search}
        />
        <GenreFilter selected={genre} onSelect={setGenre} />
        <FlatList
          key={columns}
          numColumns={columns}
          data={visibleAlbums}
          keyExtractor={(album) => album.id}
          renderItem={({ item }) => (
            <AlbumCard
              album={item}
              inCart={cart.includes(item.id)}
              onToggleCart={toggleCart}
            />
          )}
          ListEmptyComponent={
            <Text style={styles.empty}>
              Aucun album ne correspond à votre recherche.
            </Text>
          }
          contentContainerStyle={styles.list}
        />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: "#F2F3F9" },
  container: {
    flex: 1,
    width: "100%",
    maxWidth: 1280,
    alignSelf: "center",
    paddingTop: 8,
  },
```

```
  header: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    flexWrap: "wrap",
    gap: 8,
    paddingHorizontal: 16,
    paddingVertical: 8,
  },
  heading: { fontSize: 26, fontWeight: "800" },
  cart: { fontSize: 16, fontWeight: "600" },
  search: {
    marginHorizontal: 16,
    marginVertical: 8,
    padding: 12,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: "#D9DCEA",
    backgroundColor: "#fff",
    fontSize: 16,
  },
  list: { padding: 8 },
  empty: { margin: 8, fontSize: 16 },
});
```